AIRFLOW ASSIGNMENT 2

Project Title: Airflow DAG for Multi-Input Processing
Name: Parth Shah

---

1. Objective

The objective of this project is to:

- Design a DAG that accepts multiple inputs
- Process inputs using Python tasks
- Display and analyze the inputs
- Implement a simple use case based on user data

---

2. Input Parameters

The DAG accepts the following inputs:

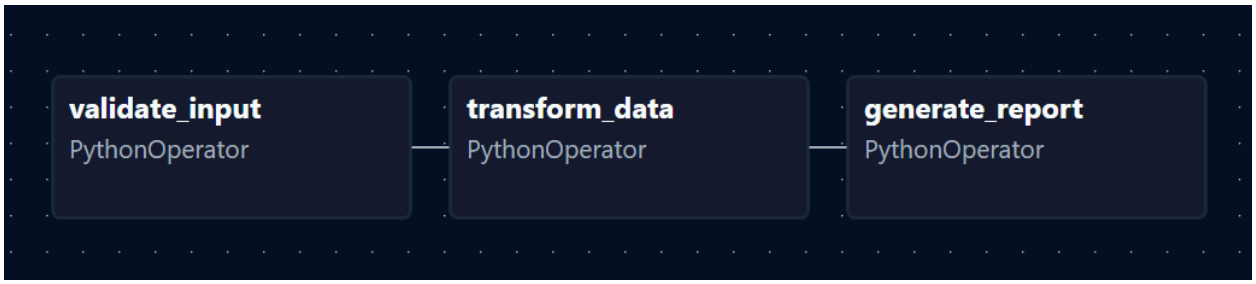
Parameter	Type	Description
name	String	User name
age	Integer	User age
height	Float	User height
is_student	Boolean	Student status
hobbies	List	List of hobbies
profile	Dict	Additional user details
run_date	String	Execution date

Code -

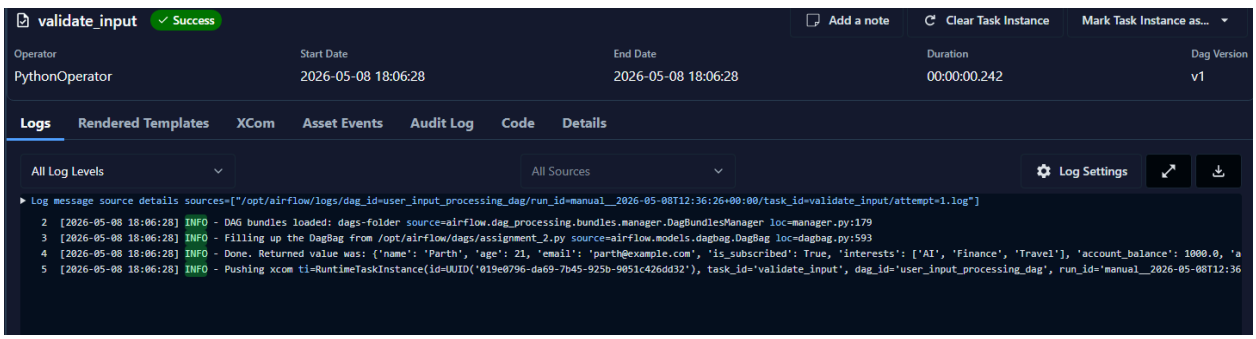
```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Simulated user input
USER_INPUT = {
    "name": "Parth",
    "age": 21,
    "email": "parth@example.com",
    "is_subscribed": True,
    "interests": ["AI", "Finance", "Travel"],
    "account_balance": 1,
    "address": {
        "city": "Ahmedabad",
        "state": "Gujarat",
        "zip": "380013"
    }
}
# Task 1: Validate inputs
def validate_input(**context):
    data = USER_INPUT

    assert isinstance(data["name"], str)
    assert isinstance(data["age"], int)
    assert isinstance(data["email"], str)
    assert isinstance(data["is_subscribed"], bool)
    assert isinstance(data["interests"], list)
    assert isinstance(data["account_balance"], float)
    assert isinstance(data["address"], dict)
    return data

# Task 2: Transform data
def transform_data(**context):
    data = context["ti"].xcom_pull(task_ids="validate_input")
    transformed = {
        "name_upper": data["name"].upper(),
        "age_next_year": data["age"] + 1,
        "email_domain": data["email"].split("@")[1],
        "premium_user": data["is_subscribed"] and data["account_balance"] > 1000,
        "interest_count": len(data["interests"]),
        "net_worth_category": "High" if data["account_balance"] > 1000 else "Low",
        "city": data["address"]["city"]
    }
    return transformed

# Task 3: Generate summary report
def generate_report(**context):
    data = context["ti"].xcom_pull(task_ids="transform_data")

    report = f"""
    USER SUMMARY REPORT

    Name: {data['name_upper']}
    Next Year Age: {data['age_next_year']}
    Email Domain: {data['email_domain']}
    Premium User: {data['premium_user']}
    Interest Count: {data['interest_count']}
    Net Worth: {data['net_worth_category']}
    City: {data['city']}
    """
    print(report)
    return report

# DAG Definition
with DAG(
    dag_id="user_input_processing_dag",
    start_date=datetime(2024, 1, 1),
    schedule_interval=None,
    catchup=False
) as dag:

    task_validate = PythonOperator(
        task_id="validate_input",
        python_callable=validate_input
    )
    task_transform = PythonOperator(
        task_id="transform_data",
        python_callable=transform_data
    )
    task_report = PythonOperator(
        task_id="generate_report",
        python_callable=generate_report
    )
    task_validate >> task_transform >> task_report
```

4. Screenshots

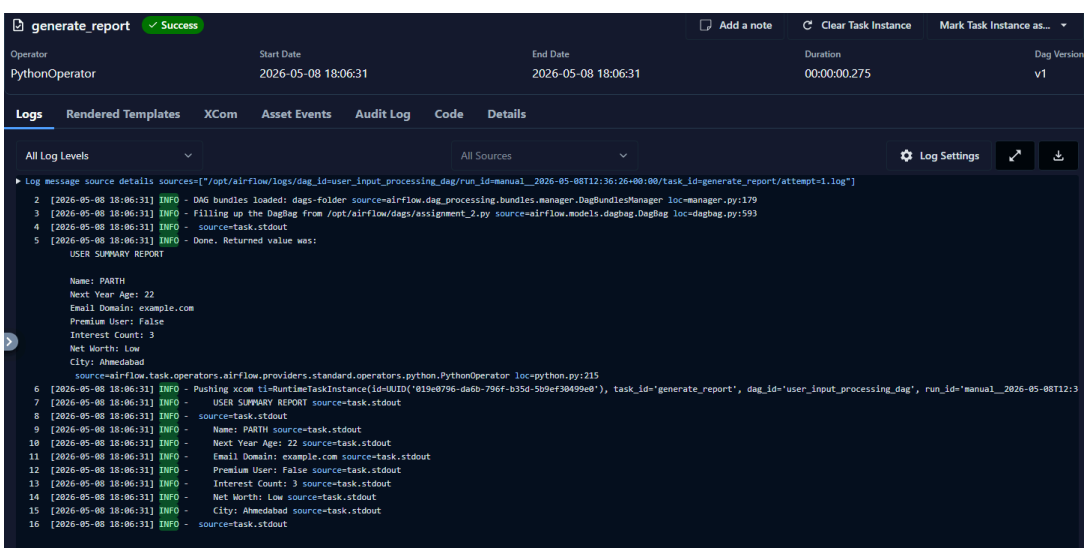
4.1 DAG Graph View



4.2 DAG Trigger with Inputs



4.3 Task Execution Logs



5. Working Explanation

- The DAG is triggered manually using input parameters.
- Inputs are passed using dag_run.conf.
- The first task reads and prints inputs.
- The second task processes and formats user data.
- The third task applies decision logic based on age.

---

6. Use Case

This DAG can be used in:

- User data processing systems
- Profile analysis pipelines
- Basic ETL workflows
- Input validation systems